

Names of the AIs used:

Claude - Sonnet 5 Medium

ChatGPT - Free

Access method:

Simply entered each name into a google search query individually and used their respective websites.

Prompt used:

"Write a program in C following the instructions below. The program will prioritize emails for a busy company CEO.

- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.

- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.

- You must create functions from scratch. Do not include pre-existing heap modules.

- Test File format:

Command:

EMAIL <sender category>,

<subject line>,

<date>

Description:

The emails in the CEO's Inbox should be placed in queue

based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

Command:

NEXT

Description:

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

Command:

READ

Description:

CEO has read email and dealt with it

Command:

COUNT

Description:

Display current count of unread emails

• The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

• Sample Test File:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson, Try our product, 12-19-2024

EMAIL Boss, Important, 12-20-2024

EMAIL Subordinate, How do I handle this?, 12-25-2024

EMAIL ImportantPerson, Health Insurance Enrollment, 12-31-2024

EMAIL Boss, Never Mind, 01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

- Your program should display this:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read"

Claude Code (unimproved - handmade comments on improved ver linked in GitHub):

```
//claude.c contained in file 3201410_Louis_Assignment_2
//C program that prioritizes emails for a company CEO
//Creation Date: 9:05AM, Thursday, September 17, 2026
//Inputs: direct inputs with bash command "./claude < testfile.txt"
/*Outputs: The program outputs email sender, how many emails to be read, subject, date, and
next email based on the commands used in the text file that is inputted.*/
//Collaborators: Claude - Sonnet 5 Medium
//Note: None of the code below is original, it was all generated by Claude - Sonnet 5 Medium
//Note2: All comments are human-made by the Author below.
//Author: Joshua Louis

#include <stdio.h> //standard input/output library
#include <stdlib.h> //standard function library for memory allocation and process control
#include <string.h> //includes functions for working with strings
#include <ctype.h> //provides functions for classifying and modifying characters

#define MAX_FIELD 256 //preprocessor macro, replaces every MAX_FIELD with the number 256

typedef struct { //email record
    char category[MAX_FIELD]; //sender category, e.g. "Boss"
    char subject[MAX_FIELD]; //subject line (may contain spaces)
    char date[MAX_FIELD]; //date string, MM-DD-YYYY, as given
    int priorityRank; //numeric rank derived from category
    long dateValue; //YYYYMMDD style value for comparison
} Email;

static int categoryRank(const char *category) { //Returns a numeric rank for a sender
category. Higher = more urgent.
    if (strcmp(category, "Boss") == 0) return 5; //
    if (strcmp(category, "Subordinate") == 0) return 4; //
    if (strcmp(category, "Peer") == 0) return 3; //
    if (strcmp(category, "ImportantPerson") == 0) return 2; //
    if (strcmp(category, "OtherPerson") == 0) return 1; //
    return 0; //unknown category - treat as lowest priority
}

/* Parses "MM-DD-YYYY" into a comparable integer YYYYMMDD.
* Larger value == newer date == should be read first (within a tie). */
static long parseDateValue(const char *date) { //
    int mm = 0, dd = 0, yyyy = 0; //
    sscanf(date, "%d-%d-%d", &mm, &dd, &yyyy); //
    return ((long)yyyy * 10000L) + ((long)mm * 100L) + (long)dd; //
}
```

```

/* Fills in priorityRank and dateValue for an Email after its
 * category/date strings have been set. */
static void computeEmailKeys(Email *e) { //
    e->priorityRank = categoryRank(e->category); //
    e->dateValue = parseDateValue(e->date); //
}

/* Returns > 0 if a has strictly higher priority than b, < 0 if lower,
 * and 0 if they are equal in priority (same category and date). */
static int compareEmails(const Email *a, const Email *b) { //
    if (a->priorityRank != b->priorityRank) { //
        return a->priorityRank - b->priorityRank; //
    }
    /* Same category: newer date (larger dateValue) wins. */
    if (a->dateValue != b->dateValue) { //
        return (a->dateValue > b->dateValue) ? 1 : -1; //
    }
    return 0; //
}

/* ----- */
/* MaxHeap - list (array) based implementation, built from scratch */
/* ----- */

typedef struct {
    Email *data;    //dynamically allocated array of Emails
    int size;       //number of emails currently stored
    int capacity;   //allocated capacity of data[]
} MaxHeap;

#define HEAP_INITIAL_CAPACITY 16 //

static void heapInit(MaxHeap *h) { //
    h->capacity = HEAP_INITIAL_CAPACITY; //
    h->size = 0; //
    h->data = (Email *)malloc(sizeof(Email) * h->capacity); //
    if (h->data == NULL) { //
        fprintf(stderr, "Fatal: could not allocate heap memory.\n"); //
        exit(EXIT_FAILURE); //
    }
}

static void heapFree(MaxHeap *h) { //
    free(h->data); //
    h->data = NULL; //
    h->size = 0; //
    h->capacity = 0; //
}

static void heapGrowIfNeeded(MaxHeap *h) { //
    if (h->size >= h->capacity) { //
        h->capacity *= 2; //
    }
}

```

```

    Email *newData = (Email *)realloc(h->data, sizeof(Email) * h->capacity); //
    if (newData == NULL) { //
        fprintf(stderr, "Fatal: could not grow heap memory.\n"); //
        exit(EXIT_FAILURE); //
    }
    h->data = newData; //
}
}

static void swapEmails(Email *a, Email *b) { //
    Email tmp = *a; //
    *a = *b; //
    *b = tmp; //
}

static int parentIndex(int i) { return (i - 1) / 2; } //
static int leftChildIndex(int i) { return 2 * i + 1; } //
static int rightChildIndex(int i) { return 2 * i + 2; } //

/* Moves the element at index i up until the max-heap property holds. */
static void siftUp(MaxHeap *h, int i) { //
    while (i > 0) { //
        int p = parentIndex(i); //
        if (compareEmails(&h->data[i], &h->data[p]) > 0) { //
            swapEmails(&h->data[i], &h->data[p]); //
            i = p; //
        } else { //
            break; //
        }
    }
}

/* Moves the element at index i down until the max-heap property holds. */
static void siftDown(MaxHeap *h, int i) { //
    for (;;) { //
        int left = leftChildIndex(i); //
        int right = rightChildIndex(i); //
        int largest = i; //

        if (left < h->size && compareEmails(&h->data[left], &h->data[largest]) > 0) { //
            largest = left; //
        }
        if (right < h->size && compareEmails(&h->data[right], &h->data[largest]) > 0) { //
            largest = right; //
        }
        if (largest == i) { //
            break; //
        }
        swapEmails(&h->data[i], &h->data[largest]); //
        i = largest; //
    }
}

```

```

/* Inserts a new email into the heap. */
static void heapPush(MaxHeap *h, Email e) { //
    heapGrowIfNeeded(h); //
    h->data[h->size] = e; //
    siftUp(h, h->size); //
    h->size++; //
}

/* Returns a pointer to the highest priority email without removing it.
 * Returns NULL if the heap is empty. */
static const Email *heapPeek(MaxHeap *h) { //
    if (h->size == 0) return NULL; //
    return &h->data[0]; //
}

/* Removes and returns the highest priority email.
 * Returns 1 on success (out is filled), 0 if the heap was empty. */
static int heapPop(MaxHeap *h, Email *out) { //
    if (h->size == 0) return 0; //

    if (out != NULL) {
        *out = h->data[0];
    }

    h->size--;
    h->data[0] = h->data[h->size];
    if (h->size > 0) {
        siftDown(h, 0);
    }
    return 1;
}

static int heapCount(MaxHeap *h) {
    return h->size;
}

/* ----- */
/* Input parsing helpers */
/* ----- */

/* Strips a trailing newline/carriage-return (and trailing spaces) from
 * a line read via fgets. */
static void stripLineEnding(char *line) {
    size_t len = strlen(line);
    while (len > 0 && (line[len - 1] == '\n' || line[len - 1] == '\r')) {
        line[--len] = '\0';
    }
}

/* Trims leading/trailing whitespace from s, in place. */
static void trimWhitespace(char *s) {

```

```

size_t len = strlen(s);
size_t start = 0;
while (start < len && isspace((unsigned char)s[start])) start++;

size_t end = len;
while (end > start && isspace((unsigned char)s[end - 1])) end--;

size_t newLen = end - start;
if (start > 0) {
    memmove(s, s + start, newLen);
}
s[newLen] = '\0';
}

/* Parses the remainder of an EMAIL command line (everything after
 * "EMAIL ") into an Email struct. Fields are comma-delimited:
 * <category>,<subject>,<date>
 * The subject may contain spaces but not commas. Returns 1 on success,
 * 0 if the line could not be parsed into exactly three fields. */
static int parseEmailFields(const char *rest, Email *e) {
    char buffer[1024];
    strncpy(buffer, rest, sizeof(buffer) - 1);
    buffer[sizeof(buffer) - 1] = '\0';

    char *firstComma = strchr(buffer, ',');
    if (firstComma == NULL) return 0;
    char *secondComma = strchr(firstComma + 1, ',');
    if (secondComma == NULL) return 0;

    *firstComma = '\0';
    *secondComma = '\0';

    char *category = buffer;
    char *subject = firstComma + 1;
    char *date = secondComma + 1;

    trimWhitespace(category);
    trimWhitespace(subject);
    trimWhitespace(date);

    strncpy(e->category, category, MAX_FIELD - 1);
    e->category[MAX_FIELD - 1] = '\0';
    strncpy(e->subject, subject, MAX_FIELD - 1);
    e->subject[MAX_FIELD - 1] = '\0';
    strncpy(e->date, date, MAX_FIELD - 1);
    e->date[MAX_FIELD - 1] = '\0';

    computeEmailKeys(e);
    return 1;
}

/* ----- */

```

```

/* Command handlers */
/* ----- */

static void handleEmailCommand(MaxHeap *h, const char *line) {
    /* line begins with "EMAIL " (6 characters) */
    const char *rest = line + 6;
    Email e;
    if (parseEmailFields(rest, &e)) {
        heapPush(h, e);
    }
    /* Malformed EMAIL lines are silently ignored, per input assumption
       that fields will be well-formed. */
}

static void handleNextCommand(MaxHeap *h) {
    const Email *top = heapPeek(h);
    if (top == NULL) {
        printf("No emails to read.\n");
        return;
    }
    printf("Next email:\n");
    printf("Sender: %s\n", top->category);
    printf("Subject: %s\n", top->subject);
    printf("Date: %s\n", top->date);
}

static void handleReadCommand(MaxHeap *h) {
    /* Pops (deletes) the highest priority email without displaying it,
       whether or not it was previously shown via NEXT. If the heap is
       empty, nothing happens. */
    heapPop(h, NULL);
}

static void handleCountCommand(MaxHeap *h) {
    int count = heapCount(h);
    printf("There are %d emails to read.\n", count);
}

int main(void) {
    MaxHeap heap;
    heapInit(&heap);

    char line[2048];
    while (fgets(line, sizeof(line), stdin) != NULL) {
        striplineEnding(line);

        /* Skip blank lines */
        if (line[0] == '\0') continue;

        if (strncmp(line, "EMAIL ", 6) == 0) {
            handleEmailCommand(&heap, line);
        }
    }
}

```

```

    } else if (strcmp(line, "NEXT") == 0) {
        handleNextCommand(&heap);
    } else if (strcmp(line, "READ") == 0) {
        handleReadCommand(&heap);
    } else if (strcmp(line, "COUNT") == 0) {
        handleCountCommand(&heap);
    }

}

heapFree(&heap);
return 0;
}

```

Correctness: The program was tested against the sample file and matched the expected output exactly. It also handles the three edge cases stated in the assignment (empty-queue **NEXT/READ**, repeated **NEXTs**, and repeated **READs**) without crashing, and weird **EMAIL** lines are dropped rather than corrupting the heap.

Execution Time: Both **heapPush** and **heapPop** are bounded by the heap's height, giving $O(\log n)$ per operation rather than any nested loop scaling with n^2 . Processing all commands over n emails runs in $O(m \log n)$, with m being the number of commands in the text.

Space Complexity: Emails are stored in a single array that doubles when full, giving $O(n)$ total space with no recursion or auxiliary structures that scale with input. A part that could be improved is the 256-bytes used regardless of the length of the string.

Maintainability: Every function has descriptive names and no magic numbers, but weird/wrong input fails silently with no message, leaving debugging a little harder.

ChatGPT Code:

```
//chatgpt.c contained in file 3201410_Louis_Assignment_2
//C program that prioritizes emails for a company CEO
//Creation Date: 9:05AM, Thursday, September 17, 2026
//Inputs: direct inputs with bash command "./claude < testfile.txt"
/*Outputs: The program outputs email sender, how many emails to be read, subject, date, and
next email based on the
commands used in the text file that is inputted.*/
//Collaborators: ChatGPT - Free
//Note: None of the code below is original, it was all generated by ChatGPT - Free
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_SUBJECT 500
#define MAX_LINE 600

/*
    Each email has:
    - sender category
    - subject
    - date
    - priority based on sender category
    - date value used to compare emails
*/
typedef struct {
    char sender[30];
    char subject[MAX_SUBJECT];
    char date[11];

    int priority;
    int dateValue;
} Email;

/*
    List-based MaxHeap.
    The heap is stored in a dynamically allocated array.
*/
typedef struct {
    Email *emails;
    int size;
    int capacity;
} MaxHeap;

/* -----
    Convert sender category to its priority.

    Higher number = higher priority.
    Boss           = 5
    Subordinate    = 4
    Peer           = 3
*/>
```

```

    ImportantPerson = 2
    OtherPerson     = 1
    ----- */
int getPriority(const char *sender)
{
    if (strcmp(sender, "Boss") == 0)
        return 5;
    else if (strcmp(sender, "Subordinate") == 0)
        return 4;
    else if (strcmp(sender, "Peer") == 0)
        return 3;
    else if (strcmp(sender, "ImportantPerson") == 0)
        return 2;
    else
        return 1;
}

/* -----
   Convert MM-DD-YYYY into a number that can be compared.

   YYYYMMDD is used so that larger values represent newer
   dates.
   ----- */
int getDateValue(const char *date)
{
    int month, day, year;

    sscanf(date, "%d-%d-%d", &month, &day, &year);

    return year * 10000 + month * 100 + day;
}

/* -----
   Return 1 if email1 should appear before email2.
   Return 0 otherwise.

   First compare sender priority.
   If priorities are equal, compare dates.
   Newer date gets higher priority.
   ----- */
int higherPriority(Email email1, Email email2)
{
    if (email1.priority > email2.priority)
        return 1;

    if (email1.priority < email2.priority)
        return 0;

    /*
       Same sender category:

```

```

        newest email gets priority.
    */
    if (email1.dateValue > email2.dateValue)
        return 1;

    return 0;
}

/* -----
   Initialize the heap.
   ----- */
void initializeHeap(MaxHeap *heap)
{
    heap->capacity = 10;
    heap->size = 0;

    heap->emails = malloc(heap->capacity * sizeof>Email));

    if (heap->emails == NULL)
    {
        printf("Memory allocation failed.\n");
        exit(1);
    }
}

/* -----
   Resize the heap when it becomes full.
   ----- */
void resizeHeap(MaxHeap *heap)
{
    heap->capacity *= 2;

    Email *newEmails =
        realloc(heap->emails, heap->capacity * sizeof>Email));

    if (newEmails == NULL)
    {
        printf("Memory allocation failed.\n");
        free(heap->emails);
        exit(1);
    }

    heap->emails = newEmails;
}

/* -----
   Swap two emails in the heap.
   ----- */
void swapEmails(Email *a, Email *b)

```

```

{
    Email temp = *a;
    *a = *b;
    *b = temp;
}

/* -----
   Move an inserted email upward until the MaxHeap property
   is restored.
   ----- */
void heapifyUp(MaxHeap *heap, int index)
{
    int parent;

    while (index > 0)
    {
        parent = (index - 1) / 2;

        if (higherPriority(heap->emails[index],
                           heap->emails[parent]))
        {
            swapEmails(&heap->emails[index],
                       &heap->emails[parent]);

            index = parent;
        }
        else
        {
            break;
        }
    }
}

/* -----
   Insert an email into the MaxHeap.
   ----- */
void insertEmail(MaxHeap *heap, Email email)
{
    if (heap->size == heap->capacity)
        resizeHeap(heap);

    heap->emails[heap->size] = email;

    heapifyUp(heap, heap->size);

    heap->size++;
}

/* -----

```

```

    Move an email downward until the MaxHeap property is
    restored.
    ----- */
void heapifyDown(MaxHeap *heap, int index)
{
    int left;
    int right;
    int largest;

    while (1)
    {
        left = 2 * index + 1;
        right = 2 * index + 2;
        largest = index;

        if (left < heap->size &&
            higherPriority(heap->emails[left],
                          heap->emails[largest]))
        {
            largest = left;
        }

        if (right < heap->size &&
            higherPriority(heap->emails[right],
                          heap->emails[largest]))
        {
            largest = right;
        }

        if (largest != index)
        {
            swapEmails(&heap->emails[index],
                      &heap->emails[largest]);

            index = largest;
        }
        else
        {
            break;
        }
    }
}

/* -----
   Remove and return the highest-priority email.

   Returns 1 if an email was removed.
   Returns 0 if the heap is empty.
   ----- */
int removeMax(MaxHeap *heap, Email *removedEmail)
{

```

```

    if (heap->size == 0)
        return 0;

    *removedEmail = heap->emails[0];

    heap->size--;

    if (heap->size > 0)
    {
        heap->emails[0] = heap->emails[heap->size];

        heapifyDown(heap, 0);
    }

    return 1;
}

/* -----
Return the highest-priority email without removing it.

This is needed for NEXT.

Returns 1 if an email exists.
Returns 0 if the heap is empty.
----- */
int peekMax(MaxHeap *heap, Email *email)
{
    if (heap->size == 0)
        return 0;

    *email = heap->emails[0];

    return 1;
}

/* -----
Free heap memory.
----- */
void destroyHeap(MaxHeap *heap)
{
    free(heap->emails);
    heap->emails = NULL;
    heap->size = 0;
    heap->capacity = 0;
}

/* -----
Create an Email from the three input fields.
----- */

```

```

Email createEmail(char *sender, char *subject, char *date)
{
    Email email;

    strcpy(email.sender, sender);
    strcpy(email.subject, subject);
    strcpy(email.date, date);

    email.priority = getPriority(sender);
    email.dateValue = getDateValue(date);

    return email;
}

/* -----
   Main program
   ----- */
int main(void)
{
    MaxHeap heap;

    char line[MAX_LINE];

    initializeHeap(&heap);

    /*
       Used to remember the email displayed by NEXT.

       This allows:
       NEXT
       NEXT

       to display the same email both times.
    */
    Email currentEmail;

    int hasCurrentEmail = 0;

    while (fgets(line, sizeof(line), stdin) != NULL)
    {
        /*
           Remove newline characters from the input.
        */
        line[strcspn(line, "\r\n")] = '\0';

        /* -----
           EMAIL command
           ----- */
        if (strncmp(line, "EMAIL ", 6) == 0)
        {
            char *sender;

```

```

char *subject;
char *date;

/*
    The input format is:

    EMAIL sender,subject,date
*/

sender = line + 6;

subject = strchr(sender, ',');

if (subject == NULL)
    continue;

*subject = '\0';
subject++;

date = strchr(subject, ',');

if (date == NULL)
    continue;

*date = '\0';
date++;

Email email = createEmail(sender, subject, date);

insertEmail(&heap, email);

/*
    A new email does not automatically replace the
    currently displayed email. The current email
    remains the result of the most recent NEXT.
*/
}

/* -----
    NEXT command
    ----- */
else if (strcmp(line, "NEXT") == 0)
{
    /*
        If NEXT has already displayed an email and it
        has not been READ, display that same email.

        Otherwise, look at the top of the heap.
    */
    if (!hasCurrentEmail)
    {
        if (peekMax(&heap, &currentEmail))

```

```

        {
            hasCurrentEmail = 1;
        }
    }

    if (hasCurrentEmail)
    {
        printf("Next email:\n");
        printf("Sender: %s\n", currentEmail.sender);
        printf("Subject: %s\n", currentEmail.subject);
        printf("Date: %s\n", currentEmail.date);
    }
}

/* -----
   READ command
   ----- */
else if (strcmp(line, "READ") == 0)
{
    Email removedEmail;

    /*
       READ always removes the highest-priority email.

       This also handles:
       READ
       READ

       because each READ removes the current highest
       priority email independently.
    */
    if (hasCurrentEmail)
    {
        removeMax(&heap, &removedEmail);
        hasCurrentEmail = 0;
    }
    else
    {
        /*
           If NEXT has not been called, READ still needs
           to remove the highest-priority email.
        */
        removeMax(&heap, &removedEmail);
    }
}

/* -----
   COUNT command
   ----- */
else if (strcmp(line, "COUNT") == 0)
{
    printf("There are %d emails to read.\n", heap.size);
}

```

```
    }  
}  
  
destroyHeap(&heap);  
  
return 0;  
}
```

Correctness: Overall MaxHeap implementation works. Date input isn't completely validated, could cause an error.

Time Complexity: Insert and **READ** are $O(\log n)$; **NEXT** and **COUNT** are $O(1)$.

Space Complexity: $O(n)$ because the heap stores n emails.

Maintainability: Good overall structure and descriptive names, but `main()` and input parsing could be broken into smaller functions.

Selected AI:

Claude - Sonnet 5 Medium

Reason:

It is more cleanly formatted than ChatGPT, even though they both lack some comments unless otherwise instructed.

What I did to improve: - finish later

Correctness: Prompted Claude to improve code by including a message for when an EMAIL line is weird/wrong (ex. No/wrong comma place) .

Execution Time: Did not improve nor remove.

Space Complexity: Cut **MAX_FIELD** in half from 256 bytes to 128 bytes

Maintainability: Added comments to each line.